Active Learning para defeitos em pontes de betão: quantos rótulos são mesmo precisos?

Tema C (AL 1): Poucos Rótulos · **Turno:** TP2 · **Grupo:** 1 · **Data:** 29 de Outubro de 2026

Membro	Nº	Papel	Tarefa individual de Deep Learning
Fausto Cananga	2025102675	A1 — Dados e Integração	Pipeline de dados, extração de embeddings DINOv2 e verificação de fuga
Gabriela Cardoso	79295	A2 — Baseline e Teto	Classificadores lineares, determinação do teto e fine-tuning da CNN de referência
Gonçalo Vilarinho	81970	A3 — Investigador PORTA-VOZ	Simulador de Active Learning e estratégias de aquisição (BALD, BADGE, CoreSet)
Pedro Ribeiro	79354	A4 — Produto e Classes Raras	Assistente interativo em Gradio, gestão do enviesamento e classes raras
Marco Maceda	66694	A5 — Avaliação e Calibração	Métricas (ALC, F1 multi-rótulo), calibração de incerteza (ECE) e validação

§1 PROBLEMA

Utilizador. Equipa de inspeção de infraestruturas que está a montar um classificador de defeitos de pontes de betão a partir de fotografias de campo.

Decisão apoiada. Duas decisões concretas e ligadas: **(i)** que recortes anotar a seguir, dado um orçamento de anotação fixo, e **(ii)** quando parar de anotar. No uso corrente, o assistente indica também que recortes rever por serem incertos.

Porque importa. Rotular recortes com fissuras, delaminações, eflorescências, armadura exposta e corrosão é trabalho de engenheiros: lento, caro e desequilibrado — as patologias críticas (armadura exposta, corrosão) são raras e são exatamente as que não podem escapar. O Active Learning (AL) propõe gastar o orçamento onde rende mais: em vez de rotular ao acaso, o modelo escolhe os recortes de que mais precisa. **A pergunta é quantitativa e honesta:** com que fração dos rótulos uma estratégia ativa atinge 95% do desempenho do treino completo, face à amostragem aleatória — e isso vale sobretudo nas classes raras?

Método não lecionado. As estratégias de aquisição de AL e o seu protocolo de simulação. O uso de **características congeladas de um modelo fundacional (DINOv2)** é o que torna cada ronda barata o suficiente para repetir com 3 seeds e obter intervalos de confiança.

§2 DADOS

CODEBRIM (Mundt et al., CVPR 2019; projeto AEROBI; Zenodo `10.5281/zenodo.2620293`), variante `CODEBRIM_classification_dataset` — recortes já divididos em treino/validação/teste.

- **Contagens medidas** no dataset: **7 677 recortes utilizáveis** de **1 522 imagens de origem** — treino **6 438** · validação **611** · teste **628** (52 recortes com vetor todo a zero excluídos). **1 990 recortes (25,9%)** têm **≥2 defeitos**: é por isso que o problema é multi-rótulo.
- **Frequência por classe** (recortes, treino/total): fissura **2 208 / 2 507** · delaminação **1 608 / 1 898** · corrosão **1 263 / 1 559** · armadura exposta **1 215 / 1 507** · **eflorescência 543 / 833**. A mais **rara** é a eflorescência (**8,4%** do treino); as mais **críticas** para a segurança são a armadura exposta e a corrosão. O trabalho trata as duas coisas, que não são a mesma.
- **Divisão oficial:** validação e teste **balanceados por desenho** (~150 por classe); o **treino não** é. Agrupamento por **imagem**, verificado por script: **0 imagens em duas partições**.
- **Licença "Other (Non-Commercial)"** — uso académico, citação obrigatória; os dados **não** são versionados no repositório (`data/` está no `.gitignore`).
- **Unidade de agrupamento: a imagem de origem.** Recortes da mesma imagem têm de ficar na mesma partição. Usamos a **divisão oficial**, que já o garante (a confirmar com contagens próprias); **teste oficial congelado**; **pool = treino oficial**; validação oficial só para seleção de hiperparâmetros.
- Extensão opcional: **SegCODEBRIM** (Jaziri et al., WACV 2024, MIT) para AL em segmentação de fissuras.

Correção importante face à primeira proposta. O CODEBRIM é um dataset de **classificação de recortes multi-rótulo**, não de deteção: **mAP@0,5:0,95 não se aplica** aqui. É por isso que a métrica principal é Macro-F1 sobre recortes.
Verificado no próprio dataset (`scripts/contar_codebrim.py`, md5 do zip conferido): contagens por classe e partição, 25,7% de recortes multi-rótulo e **zero** imagens de origem em duas partições. O que falta é só a inspeção visual de 5 recortes por classe. Comparação com alternativas em `DATASETS.md`.

Tabela 2.1 — Análise do problema (validada em grupo na PL06)

Pergunta	Resposta do grupo
Quem usa a aplicação e que decisão apoia?	Equipa de inspeção de infraestruturas. Decisão: que recortes anotar a seguir e quando parar de anotar; no uso corrente, que recortes rever por serem incertos.
Dataset, origem e licença	CODEBRIM (classificação de recortes multi-rótulo, 5 classes + fundo), divisão oficial. Não comercial; citar Mundt et al. (2019); contagens reais por classe e partição no README.
Nº de imagens/anotações e tipo de anotação	**7 677 recortes utilizáveis** de **1 522 imagens** (treino 6 438 · val 611 · teste 628) · rótulo **multi-rótulo** (25,9% com ≥2 defeitos, 29 combinações). Mais **rara**: eflorescência (833; 543 no treino); mais **críticas**: armadura exposta (1 507) e corrosão (1 559).
Unidade de agrupamento e como dividir	**Imagem de origem** (recortes da mesma imagem na mesma partição); divisão oficial confirmada; teste oficial congelado; pool = treino oficial; validação só para hiperparâmetros.
Métrica principal e porquê	**Macro-F1 multi-rótulo** (threshold por classe escolhido na validação) **em função da fração de rótulos** — as classes raras pesam o mesmo que as frequentes. Resumos: **ALC** e **rótulos para 95% do teto**, com IC 95% por bootstrap de imagens de teste e por seed.
Métricas secundárias	AP multi-rótulo; F1 nas classes raras; **ECE**; composição do conjunto rotulado por ronda (fração de cada classe); tempo por ronda.
Baseline (método lecionado)	**Amostragem aleatória** com o mesmo orçamento (a única comparação justa para AL) e **modelo com 100% dos rótulos** = teto. Classificador: cabeça linear sobre **DINOv2** congelado (barato) + CNN pré-treinada afinada (confirmação).
Método avançado e artigo de referência	Aquisição por **incerteza multi-rótulo** (entropia média/margem), **BALD** (MC Dropout na cabeça), **CoreSet** (k-center greedy em embeddings) e **BADGE** (embeddings de gradiente + k-means++) — Sener & Savarese 2018; Ash et al. 2020; Gal et al. 2017; survey Settles 2009.
Estimativa de custo computacional	Extração DINOv2 de todos os recortes **uma vez** (minutos em GPU T4); cada ronda com cabeça linear/MLP: segundos a minutos → 5 estratégias × 3 seeds × 14 rondas é viável. A CNN afinada é o gargalo: só para 2 estratégias × 3 seeds, e só na confirmação. Medição em §5.

Baseline e teto — nota de coerência. A PL08 §2.3 recomenda, para o tema C, "CNN pré-treinada (EfficientNet) com fine-tuning, 100% dos rótulos" como **baseline do projeto**; o Protocolo do Tema C define o **teto** com `ResNet-50` afinada e a **cabeça linear sobre DINOv2** como classificador principal. Declaramos os dois papéis e não os confundimos: **baseline = aleatória com o mesmo orçamento** (a comparação que interessa ao AL) · **teto = 100% dos rótulos** (cabeça linear e, na confirmação, ResNet-50 afinada).

§3 MÉTODOS, HIPÓTESES E PROTOCOLO DE SIMULAÇÃO

Simulador pool-based. `pool = treino oficial`; `L` inicial rotulado por seed; em cada ronda o classificador é treinado **de raiz** com hiperparâmetros congelados, prevê probabilidades no pool `U`, adquire `K` recortes e o **oráculo** (o próprio dataset) revela os rótulos verdadeiros.

Parâmetro	Valor fixado
`L` **(conjunto inicial)**	**5%** do pool, **estratificado**, 3 seeds (fixa também a inicialização)
`K` **(por ronda)**	**2,5%** do pool
Orçamento total	**40%** do pool → **14 rondas**
Seeds	3, iguais para todas as estratégias
Hiperparâmetros	escolhidos **uma vez** na validação com a estratégia **aleatória** e congelados; re-treino de raiz por ronda (warm start só na ResNet-50, declarado)
Teste	oficial, congelado; avaliado em todas as rondas mas **lido só depois do commit do** `PROTOCOLO.md`

Hipóteses. **H1** — pelo menos uma estratégia ativa atinge 95% do Macro-F1 do teto com **menos rótulos** do que a aleatória, com IC (3 seeds) **não sobreposto** no número de rótulos necessários (a fração é reportada, não imposta). **H2** — o ganho **concentra-se nas classes escassas e nas críticas**, que aqui **não são as mesmas**: o F1 da **eflorescência** (a mais rara, 833 caixas) e o da **armadura exposta + corrosão** (as críticas) a 20% de rótulos são superiores com AL do que com aleatória (IC da diferença emparelhada por seed a excluir 0). Se o ganho aparecer só nas críticas e não na rara, isso é um resultado — e é o mais interessante dos dois. **H3** — as estratégias de **diversidade/híbridas** (CoreSet, BADGE) superam a **incerteza pura** nas primeiras rondas (*cold start*) e a diferença desaparece com o orçamento. **H4** (exploratória) — o modelo treinado com amostras ativas está **pior calibrado** (ECE maior) do que o treinado com amostras aleatórias, para o mesmo orçamento.

Reprodutibilidade (Munjal et al. 2022): `L` e `K` fixos e iguais para todas as estratégias; as mesmas 3 seeds; `splits.json` e as listas adquiridas por ronda com **hash**; confronto com as verificações de sanidade V1–V3 e V6.

§4 APLICAÇÃO — ASSISTENTE DE ANOTAÇÃO (MOCKUP)

Gradio, `app/app.py`. Entrada: lote de recortes não rotulados (ou imagem inteira fatiada), escolha da estratégia e orçamento da ronda `K`.

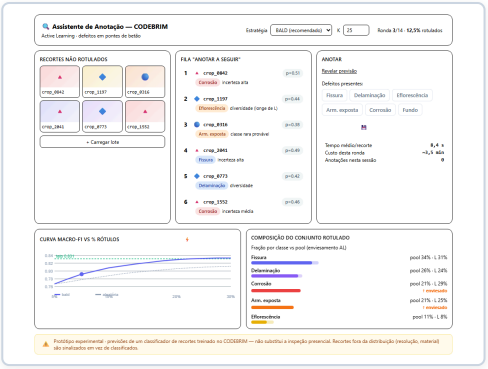


Figura 1 — *Assistente de anotação interativo (Gradio) com re-treino em tempo real e visualização da composição de classes rotuladas.*

O re-treino da cabeça em segundos é o que fecha o ciclo: o anotador vê a curva melhorar enquanto trabalha. A composição do conjunto rotulado (fração por classe) é mostrada para tornar visível o **enviesamento que o próprio AL cria**.

§5 PLANO, PAPÉIS E CUSTO

Semana	Marco	Responsável
6 (22/10)	Repositório, tabela 2.1, guia de segmentação, app v0	A1 + todos
7 (**29/10**)	**P2-D1** — esta proposta + pitch; hipóteses, `L` / `K` /orçamento fixados; regra de paragem; acordo de autoria	A3 (pitch) · A1 · todos
8 (05/11)	`src/dados.py` com verificação de fuga; baseline registada em `resultados.csv`	A1 · A2
9 (12/11)	Entropia/margem e BALD; CoreSet e BADGE; `PROTOCOLO.md` **fechado**; `avaliar.py`	A3 · A5
10 (**17/11**)	**P2-D2** — Seminário I (25 min + 5 min), porta-voz A3	A3 · todos
11–12 (03/12)	Assistente de anotação a funcionar; **P2-D3** (tag `v1-D3`); revisão cruzada	A1 · A4 · A2 ↔ A3 , A4 ↔ A5
13–14 (17/12)	ECE, Grad-CAM, corrida única no teste; **P2-D4** (Tabelas 1–3)	A5 · A4 · todos
15 (04/01)	Relatório, repositório limpo, tag `v2-final` ; **P2-D5** 05/01 e defesa 07/01	todos

Estimativa computacional (fórmula fixa, valores a medir na PL07 — ver `PENDENTES.md` §5, `P7.8`):

Fase	Cálculo	Medir
Extração DINOv2	`N_recortes × t_forward(ViT-S/14)` — uma só vez	`t_forward` e nº de recortes reais
Rondas (cabeça linear)	`5 estratégias × 3 seeds × 14 rondas = 210` treinos de cabeça + 210 inferências no pool	tempo por treino e por inferência
BADGE	acrescenta `|U| × C` embeddings de gradiente por ronda (barato para cabeça linear)	tempo por ronda do BADGE
Confirmação	`2 estratégias × 3 seeds × 14 rondas = 84` fine-tunes de ResNet-50	tempo/época em subconjunto × épocas
Avaliação	bootstrap (10 000 réplicas) × condições	tempo de um bootstrap

O conjunto de rondas com cabeça linear cabe num Colab gratuito; a confirmação com ResNet-50 é a única parte que exige contenção (2 estratégias, não 5).